

Viva Cheat Sheet — Dental Management System

Quick-reference Q&A for project defense

1. Project-Specific

Q: What tech stack did you use?

MERN: MongoDB (database), Express (backend framework), React (frontend), Node.js (JS runtime). Plus Mongoose (ODM), JWT + bcrypt (auth), Tailwind CSS (styling), Recharts (charts), jsPDF (PDF export).

Q: Difference between Admin and Staff?

Same User model, different 'role' field. Staff can add/edit records but not delete them or manage pricing/staff. Admin has full control: delete patients/invoices, manage the service price list, and the Staff Management page. Enforced on the backend via an adminOnly middleware, not just hidden in UI.

Q: What is Axios?

A library for making HTTP requests from React to the Express API. I created one shared instance with a request interceptor that auto-attaches the JWT token to every call, so pages don't repeat that code.

Q: How does login work end-to-end?

User submits email/password -> backend finds user, compares hashed password with bcrypt -> if valid, signs a JWT with user id/role -> frontend stores token in localStorage -> Axios attaches it as 'Authorization: Bearer token' on every future request -> backend middleware verifies it each time.

Q: How are Patients, Dentists, Appointments related in MongoDB?

Appointment stores the Patient's and Dentist's _id as a reference (not their full data). Mongoose's .populate() swaps that id for the real object when reading data back.

Q: How does search/pagination work on the Patients page?

Frontend sends ?search=&gender;=&page;=&limit;= as query params. Backend builds a MongoDB regex filter, then uses .skip().limit() to fetch only that page, and also returns the total count for page numbers.

Q: How is the invoice PDF generated?

Entirely in the browser with the jsPDF library -- no server call. It draws text and a table onto a virtual PDF canvas and triggers a direct download.

Q: How does the notification bell stay updated?

The frontend polls an /unread-count endpoint every 15 seconds with setInterval. New appointments and invoices auto-create a Notification document on the backend the moment they're created.

Q: Why split into backend and frontend folders?

Separation of concerns: backend only handles data + business logic + security; frontend only handles what the user sees. They communicate purely over HTTP/JSON, so either could be swapped independently.

2. Core Web / Backend Concepts

Q: What is MongoDB and why not MySQL?

MongoDB is a NoSQL, document-based database -- data is stored as JSON-like documents instead of rows in tables. Chosen because records like invoices (variable line items) fit naturally without needing multiple joined tables.

Q: What is Mongoose?

A library on top of MongoDB that lets you define Schemas -- a blueprint of fields, types, and validation rules (e.g. required, enum). MongoDB itself has no fixed structure without it.

Q: What is Express?

A minimal Node.js web framework for defining routes (app.get/post/put/delete) and middleware, instead of writing raw HTTP server code by hand.

Q: What is REST / a REST API?

An API style where HTTP methods map to actions on resources: GET=read, POST=create, PUT=update, DELETE=remove, using URLs like /api/patients/:id.

Q: What is JWT?

JSON Web Token -- a signed token issued at login containing the user's id/role. It lets the server verify 'who is this request from' without storing server-side sessions (stateless auth).

Q: Why hash passwords? What is bcrypt?

So a leaked database never exposes real passwords. Bcrypt is one-way: you can check a typed password against a hash, but can never reverse a hash back into the original text.

Q: What is middleware? Give an example from your project.

Code that runs before the route handler. My auth.js checks the JWT on every protected route and calls next() to continue, or returns 401 immediately if invalid.

Q: What is CORS and why did you need it?

Cross-Origin Resource Sharing. Frontend (port 5173) and backend (port 5000) are different origins; browsers block cross-origin requests by default, so the cors package explicitly allows it.

Q: Difference between authentication and authorization?

Authentication = proving who you are (login/JWT check). Authorization = checking what you're allowed to do once identified (the adminOnly role check).

Q: What is CRUD?

Create, Read, Update, Delete -- the four basic operations every resource in this app supports (patients, dentists, appointments, invoices, services).

Q: Difference between SQL and NoSQL?

SQL databases use fixed tables/rows with relationships via foreign keys and joins (e.g. MySQL). NoSQL (MongoDB) stores flexible JSON-like documents, better suited to nested/variable data.

Q: Difference between PUT and POST?

POST creates a new resource (e.g. new patient). PUT updates an existing one by id (e.g. /patients/:id). Both send data in the request body.

Q: What do HTTP status codes 200, 201, 400, 401, 403, 404, 500 mean?

200 OK (success), 201 Created (new resource made), 400 Bad Request (invalid input), 401 Unauthorized (no/invalid token), 403 Forbidden (valid token, not allowed), 404 Not Found, 500 Server Error.

Q: What is JSON?

JavaScript Object Notation -- a lightweight text format for structured data, like {name: 'John', age: 30}. It's how the frontend and backend exchange data.

Q: What are environment variables / .env files for?

Storing config that changes between environments (DB connection string, JWT secret, port) outside the code, so secrets aren't hardcoded or committed to version control.

Q: What is async/await and why is it used here?

Syntax for handling asynchronous operations (like a database query) that take time, without blocking other code. 'await' pauses only that function until the promise resolves. Almost every route handler uses async/await since MongoDB calls are asynchronous.

3. Frontend / React Concepts

Q: What is a React component?

A reusable, self-contained piece of UI written as a JavaScript function that returns JSX (e.g. StatCard, Modal, or a full page like Dashboard).

Q: What is JSX?

A syntax extension that lets you write HTML-like markup directly inside JavaScript, which React compiles into actual DOM elements.

Q: What is useState and useEffect?

useState holds data that can change and triggers a re-render when updated (e.g. the patients list). useEffect runs code on load or when a dependency changes -- used here to fetch data when a page opens.

Q: What is the Context API and why did you use it?

A way to share data (logged-in user, dark mode, toasts) across many components without manually passing props down every level ('prop drilling'). Used for Auth, Theme, and Toast state.

Q: Difference between props and state?

Props are data passed INTO a component from its parent (read-only). State is data a component owns and can change itself, causing it to re-render.

Q: Why Tailwind CSS instead of plain CSS?

Utility classes (e.g. bg-cyan-600, rounded-lg) let you style directly in JSX without switching files or inventing class names, and made the dark-mode variant (dark:bg-slate-900) simple to add everywhere.

Q: Is this app responsive / mobile friendly?

Yes, Tailwind's responsive prefixes (like md:grid-cols-3) are used to adjust layouts like the dashboard cards and dentist grid for smaller screens.

Q: What is client-side rendering (CSR)? Does this app use it?

The browser downloads a mostly-empty HTML page and JavaScript renders everything client-side. This app uses CSR via React/Vite -- the backend only ever returns JSON, never HTML pages.

Q: How does routing work without full page reloads?

React Router intercepts link clicks and swaps which component renders based on the URL, without asking the browser to reload -- this is called a Single Page Application (SPA).

Q: What is npm and package.json?

npm is Node's package manager for installing libraries. package.json lists a project's dependencies and scripts (like 'npm run dev') so anyone can install the exact same setup with 'npm install'.

4. Likely Curveball Questions

Q: What happens if two staff book the same slot for a dentist at once?

Currently not explicitly blocked -- an honest limitation. A future improvement would be a backend check for existing appointments at that dentist/date/time before creating a new one.

Q: How do you handle errors from the API?

Each route wraps its logic in try/catch and responds with a JSON error message and an appropriate status code; the frontend shows the error in a toast notification.

Q: Is your app secure? What security measures did you include?

Passwords hashed with bcrypt, JWT-based auth with expiry, role-based authorization on sensitive routes, file upload type/size limits via Multer, and Mongoose schema validation on all inputs.

Q: How would you deploy this app?

Backend to a Node host (e.g. Render/Railway) with MongoDB Atlas as the hosted database; frontend built with 'vite build' and deployed as static files (e.g. Vercel/Netlify), pointed at the backend URL.

Q: What would you improve if you had more time?

Real password-reset via email, appointment slot conflict checks, a patient-facing portal, audit logs, and automated tests for the API routes.

Q: Why did you choose this project?

It models a real, relatable business workflow (patients, scheduling, billing) and let me practice a full-stack MERN build end-to-end: auth, roles, relational data via references, file uploads, and reporting -- realistic skills used in real jobs.

Q: What is the difference between a model and a schema in Mongoose?

A Schema is the blueprint (field names, types, rules). A Model is what you actually use in code -- it's created FROM a schema and gives you methods like .find(), .create(), .findByIdAndUpdate().

Q: How is a file (X-ray/photo) upload handled?

Multer middleware saves the uploaded file to a local 'uploads' folder with a unique filename, then the backend returns a URL which is saved on the medical record and used to display/download it later.

Q: Why use .populate() instead of storing full patient/dentist data in every appointment?

Avoids duplicating data -- if a patient's phone number changes, it updates in one place (the Patient document) instead of needing to update every appointment that referenced it.

5. Patient Portal (Self-Service Add-on)

Q: How is the patient portal kept separate from the staff app?

Separate JWT tokens (role: 'patient' vs staff roles), a separate patientOnly middleware, a separate Axios instance with its own localStorage key, and separate React routes/pages -- a patient token can never access staff routes or vice versa.

Q: How does email sending work, and what happens if it's not configured?

Nodemailer sends via SMTP using credentials from .env. sendMail() is written to never throw -- if SMTP isn't configured or sending fails, it logs a warning and returns false, so the feature that triggered it (booking, registering) still succeeds either way.

Q: How are appointment reminder emails sent automatically?

A node-cron job runs every hour, finds Scheduled appointments 24-48 hours away that haven't had a reminder sent, emails each patient, then marks reminderSent=true so it's never sent twice.

Q: How are the PDF receipt/report and the emailed PDF the same file?

PDFKit generates the PDF into an in-memory Buffer (not a saved file). That same buffer is reused: sent directly as a download response, or attached to an outgoing email -- generated once, used twice.

Q: What happens when a patient registers if staff already added them?

The register route checks for an existing Patient record by email. If found, it links portal access to that record (adding a password) instead of creating a duplicate -- so their existing appointment/medical history is visible immediately after they sign up.